Java 21 — Virtual Threads & Structured Concurrency

Source basis: Oracle Java 21 documentation; OpenJDK JEP 444 and JEP 453

Learning goals

- Explain core concepts and when they are useful.
- Read and adapt code/query examples.
- Recognize trade-offs, common errors, and verification steps.

How to use this chapter

Read the concept first, run examples in an isolated environment, then use the official references at the end of the chapter to verify version-specific behavior. The examples are teaching samples, not production-ready systems.

1. Scope and learning outcomes

Virtual threads are lightweight Java threads managed by the JDK. They allow a thread-per-task style for applications with many concurrent operations, especially tasks that spend substantial time waiting for I/O.

This guide is intentionally limited to virtual threads, blocking I/O, task lifetime, cancellation, pinning, and structured concurrency. It is not a general Java 21 language guide.

Learning outcomes: create virtual threads, choose suitable workloads, manage executor lifecycles, understand failure propagation, and recognize the preview status of StructuredTaskScope in Java 21.

Implementation and review notes

Check interruption, timeouts, task cleanup, downstream limits, and whether the workload is I/O-bound. For StructuredTaskScope, compile with the Java 21 preview flags and verify the exact preview API.

2. Platform threads vs virtual threads

A platform thread is closely tied to an operating-system thread for its lifetime. Virtual threads are scheduled by the Java runtime over carrier platform threads and can usually unmount during supported blocking operations, freeing the carrier for other work.

Virtual threads are not faster at executing CPU instructions. Their main benefit is throughput and scalability for workloads with many concurrent waits. CPU-bound work remains constrained by available processor cores.

Virtual threads do not remove race conditions, deadlocks, database connection limits, remote-service quotas, or the need for timeouts and backpressure.

Example code / SQL

```
public class ThreadKinds {
    public static void main(String[] args) throws InterruptedException {
        Thread t = Thread.ofVirtual().name("example-vt").start(() ->
            System.out.println("virtual=" +
                Thread.currentThread().isVirtual()));
        t.join();
    }
}
```

Implementation and review notes

Check interruption, timeouts, task cleanup, downstream limits, and whether the workload is I/O-bound. For StructuredTaskScope, compile with the Java 21 preview flags and verify the exact preview API.

3. Starting virtual threads

Java 21 offers Thread.ofVirtual(), Thread.startVirtualThread(Runnable), and Executors.newVirtualThreadPerTaskExecutor(). The executor pattern is convenient when several independent tasks must be started and joined.

Keep the work unit small and self-contained. Use named threads when it helps logs and diagnostics. Use interruption-aware APIs and ensure operations have deadlines.

Example code / SQL

```
public class StartVirtual {
    public static void main(String[] args) throws InterruptedException {
        Thread t = Thread.startVirtualThread(() -> {
            System.out.println("Hello from " +
                Thread.currentThread());
        });
        t.join();
    }
}
```

Implementation and review notes

Check interruption, timeouts, task cleanup, downstream limits, and whether the workload is I/O-bound. For StructuredTaskScope, compile with the Java 21 preview flags and verify the exact preview API.

4. One virtual thread per task

A virtual-thread-per-task executor creates a new virtual thread for each submitted task. Unlike a fixed pool of platform threads, it is not intended to be a thread-count limiter.

Limit scarce downstream resources separately—for example, with a connection pool or Semaphore. Closing the executor with try-with-resources waits for submitted work to finish, so tasks should not wait forever.

Example code / SQL

```
import java.util.concurrent.*;
import java.util.stream.IntStream;

public class PerTask {
    public static void main(String[] args) {
        try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
            var futures = IntStream.range(0, 8)
                .mapToObj(i -> ex.submit(() -> {
                    Thread.sleep(50);
                    return "task " + i + " virtual=" +
                        Thread.currentThread().isVirtual();
                })).toList();
            for (var f : futures) {
                try { System.out.println(f.get()); }
                catch (Exception e) { throw new RuntimeException(e); }
            }
        }
    }
}
```

Implementation and review notes

Check interruption, timeouts, task cleanup, downstream limits, and whether the workload is I/O-bound. For StructuredTaskScope, compile with the Java 21 preview flags and verify the exact preview API.

5. Blocking I/O and service fan-out

Virtual threads are useful when a request must call multiple independent services and wait for their responses. Ordinary blocking code can remain readable while many requests are in flight.

Use client-level connection/request timeouts and propagate failures deliberately. Parallelize only independent operations; if one result is required to start another, preserve the dependency.

Do not infer production performance from a sample that only sleeps. Benchmark realistic I/O, payload sizes, service limits, and tail latency.

Example code / SQL

```
import java.util.concurrent.*;

public class FanOut {
    static String profile() throws InterruptedException {
        Thread.sleep(100); return "profile";
    }
    static String orders() throws InterruptedException {
        Thread.sleep(150); return "orders";
    }
    public static void main(String[] args) throws Exception {
        try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
            var a = ex.submit(FanOut::profile);
            var b = ex.submit(FanOut::orders);
            System.out.println(a.get() + " + " + b.get());
        }
    }
}
```

Implementation and review notes

Check interruption, timeouts, task cleanup, downstream limits, and whether the workload is I/O-bound. For StructuredTaskScope, compile with the Java 21 preview flags and verify the exact preview API.

6. Structured concurrency: the problem

Unstructured concurrency can leave child tasks running after their parent has returned. Error handling and cancellation become scattered, and a failure in one operation may not promptly stop related work.

Structured concurrency treats related subtasks as one unit of work. A scope coordinates task completion and defines what happens when a child fails or succeeds.

In Java 21, StructuredTaskScope is a preview API described by JEP 453. Preview features require preview flags and can change between Java releases.

Example code / SQL

```
// Java 21 preview API illustration
// Compile/run with --enable-preview --release 21
import java.util.concurrent.StructuredTaskScope;

class StructuredDemo {
    static String left() { return "left"; }
    static String right() { return "right"; }

    static String combine() throws Exception {
        try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
            var a = scope.fork(StructuredDemo::left);
            var b = scope.fork(StructuredDemo::right);
            scope.join();
            scope.throwIfFailed();
            return a.get() + " + " + b.get();
        }
    }
}
```

Implementation and review notes

Check interruption, timeouts, task cleanup, downstream limits, and whether the workload is I/O-bound. For StructuredTaskScope, compile with the Java 21 preview flags and verify the exact preview API.

7. Scope policies and lifecycle

ShutdownOnFailure is appropriate when every child result is required: a failure triggers shutdown of unfinished sibling tasks. ShutdownOnSuccess suits cases where one successful answer is enough, such as trying alternative providers.

A typical scope lifecycle is fork subtasks, join the scope, propagate failures, consume results, then close the scope. Always verify exact signatures and preview requirements against the Java 21 API used by the project.

Structured concurrency organizes task lifetime; it does not automatically cap database connections or remote request rates.

Implementation and review notes

Check interruption, timeouts, task cleanup, downstream limits, and whether the workload is I/O-bound. For StructuredTaskScope, compile with the Java 21 preview flags and verify the exact preview API.

8. Pinning and resource management

In Java 21, blocking while inside certain synchronized regions or native/foreign calls can pin a virtual thread to its carrier and reduce scalability under contention. Pinning does not automatically make code incorrect.

Avoid long blocking I/O while holding monitors where practical, but do not mechanically replace every synchronized block. Profile and fix cases that are frequent and costly.

Use JFR and supported JDK diagnostics to investigate pinning. Keep critical sections short and do not hold locks across remote calls unless required for correctness.

Example code / SQL

```
// Avoid holding a monitor across slow I/O when the design permits.
final Object lock = new Object();
void updateLocalState() {
    synchronized (lock) {
        // Short, in-memory state update only.
    }
}
// Perform remote I/O outside the monitor when safe.
```

Implementation and review notes

Check interruption, timeouts, task cleanup, downstream limits, and whether the workload is I/O-bound. For StructuredTaskScope, compile with the Java 21 preview flags and verify the exact preview API.

9. Observability and performance testing

Measure throughput, latency percentiles, failures, CPU, heap use, connection-pool waits, and downstream saturation. Compare equivalent workloads and include realistic concurrency and payload sizes.

Test timeouts, interruption, failure propagation, resource cleanup, and overload. A benchmark that only sleeps illustrates scheduling but is not a reliable production forecast.

Virtual threads can make it practical to represent each request as a thread, but application architecture still needs deadlines, quotas, and clear ownership.

Example code / SQL

```
// A simple diagnostic: name work and log its execution mode.
Thread.startVirtualThread(() -> {
    Thread t = Thread.currentThread();
    System.out.printf("name=%s virtual=%s%n",
        t.getName(), t.isVirtual());
});
```

Implementation and review notes

Check interruption, timeouts, task cleanup, downstream limits, and whether the workload is I/O-bound. For StructuredTaskScope, compile with the Java 21 preview flags and verify the exact preview API.

10. Review checklist and exercises

Checklist: use JDK 21; choose virtual threads for I/O-heavy concurrency; use one virtual thread per task where appropriate; set downstream limits and timeouts; propagate failures; preserve interruption; and profile representative workloads.

Exercises: launch many simulated I/O tasks; make one child fail; cap concurrent access with a Semaphore; compare throughput and latency to a platform-thread baseline; and inspect pinning under a controlled test.

Review questions: Why are virtual threads not a CPU speed-up? What does StructuredTaskScope coordinate? Why must Java 21 preview status be documented?

Implementation and review notes

Check interruption, timeouts, task cleanup, downstream limits, and whether the workload is I/O-bound. For StructuredTaskScope, compile with the Java 21 preview flags and verify the exact preview API.

Official references and further reading

These trusted official sources are the authority for version-specific details. This PDF summarizes key ideas; consult the linked references for full API specifications and additional examples.

Oracle: Virtual Threads
https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html

OpenJDK JEP 444: Virtual Threads
https://openjdk.org/jeps/444

OpenJDK JEP 453: Structured Concurrency (Preview)
https://openjdk.org/jeps/453

Java 21 Executors API
https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/Executors.html

Suggested practice checklist

• Confirm the runtime/version.
• Run the example in an isolated project or disposable database.
• Test one normal case and one failure/edge case.
• Record commands, expected output, and any assumptions.